

Narzędzia i Aplikacje Jakarta EE

Jakarta RESTful Web Services

Michał Wójcik

Usługi sieciowe

Usługi sieciowe - aplikacje klienckie i serwerowe komunikujące się ze sobą poprzez WWW z wykorzystaniem protokołu HTTP:

- interoperacyjność pomiędzy aplikacjami uruchomionymi na różnych platformach i frameworkach,
- opisy możliwe do przetwarzania przez aplikacje,
- wykorzystanie XML lub JSON,
- luźno powiązane usługi można łączyć w celu uzyskania skomplikowanych operacji.

RESTful Web Services

Podstawowe założenia:

- bezstanowe, interakcja powinna być odporna na restart serwera,
- usługi buforowania serwera aplikacji oraz innych elementów mogą być wykorzystane w celu usprawnienia wydajności, o ile elementy zwracane przez usługę nie są generowane dynamicznie i mogą być zbuforowane,
- możliwy opis za pomocą WADL lub OpenAPI,
- producent i konsument muszą obsługiwać ten sam kontekst i przesyłaną zawartość,
- mały narzut na dane, idealny dla urządzeń z ograniczonymi zasobami,
- często wykorzystywane w połączeniu z technologią AJAX.

Model dojrzałości usług REST

Poziomy dojrzałości usług (Richardson Maturity Model):

- Level 0: Swamp of POX:
  - używa protokołu HTTP jedynie jako tunelu;
- Level 1: Resources:
  - definiuje zasoby,
- Level 2: HTTP verbs:
  - wykorzystanie poprawnych metod HTTP,
- Level 3: Hypermedia controls:
  - wykorzystanie HATEOAS.

Poziom 1: wyróżnianie zasobów

Zasoby powinny być ułożone w hierarchię (rzeczowniki zamiast czasowników):

- `api/books` – wszystkie książki,
- `api/books/1` – książka o indeksie 1,
- `api/books/1/authors` – autorzy konkretnej książki,
- `api/books/1/authors/1` – autor konkretnej książki o indeksie 1.

Typy parametrów:

- path param – parametr będący elementem adresu, wskazuje na zasoby,
- query param – parametr doklejany do adresu (po znaku ?), doprecyzowują zapytanie użytkownika (aplikacji klienckiej)):
  - stronicowanie: `api/books?offset=10&limit=5` – pobranie pięciu książek począwszy od 10.
  - filtrowanie: `api/books?unread=true` - pobranie tylko nieprzeczytanych książek,
  - sortowanie: `api/books?sort=latest` - pobranie książek posortowanych po dacie wydania.

Poziom 2: operacje na zasobach

Zarządzanie zasobami poprzez operacje HTTP:

- PUT - wstawienie zasobu (aktualizacja lub stworzenie),
- GET - pobranie zasobu,
- POST - dodanie zasobu do kolekcji,
- DELETE - usunięcie zasobu.

operacja	zasób	efekt
GET	`api/books`	wszystkie książki
GET	`api/books/1`	konkretna książka
POST	`/api/books`	dodanie nowej książki
PUT	`/api/books`	zastąpienie kolekcji
PUT	`/api/books/1`	wstawienie książki
DELETE	`/api/books`	usunięcie kolekcji
DELETE	`/api/books/1`	usunięcie książki

Poziom 2: operacje na zasobach cd

Ponieważ żądanie POST nie jest idempotentne, z punktu widzenia REST powinno stosować się żądanie PUT.

Dodatkowo możliwe jest wykorzystanie PATCH do częściowej aktualizacji.

operacja	zasób	efekt
GET	`api/books`	wszystkie książki
GET	`api/books/1`	konkretna książka
PUT	`/api/books`	zastąpienie kolekcji
PUT	`/api/books/1`	wstawienie książki
PATCH	`/api/books/1`	modyfikacja książki
DELETE	`/api/books`	usunięcie kolekcji
DELETE	`/api/books/1`	usunięcie książki

POLITECHNIKA GDAŃSKA

REST jako CRUD

operacja	REST	SQL
create	PUT	INSERT
read	GET	SELECT
update	PACH	UPDATE
delete	DELETE	DELETE

Poziom 3: relacje hipertekstowe

HATEOAS - Hypermedia As The Engine Of Application State.

```
{
  "id": 2,
  "title": "Pan Lodowego Ogrodu tom 1",
  "publishDate": "2011-02-05",
  "cover": "HARD",

  "_links": {
    "self": {
      "href": "http://example.com/library/api/books/2"
    }
  },

  "_embedded": {
    "authors": [
      {
        "id": 2,
        "name": "Jarosław",
        "surname": "Grzędowicz",
        "_links": {
          "self": {
            "href": "http://example.com/library/api/authors/2"
          }
        }
      }
    ]
  }
}
```

JAX-RS

Jakarta RESTful Web Services:

- wcześniej Java API for RESTful Web Services (JAX-RS),
- standard dla tworzenia usług sieciowych zgodnych ze standardem REST na platformie Java EE,
- Jersey - implementacja referencyjna,
- RESTEasy - implementacja w serwerze WildFly,
- wymagany w ramach Java EE Web i Full profile,
- łatwa konfiguracja za pomocą adnotacji.

Aktywacja

Aktywacja JAX-RS wymaga określenia bazowego adresu dla zasobów:

```
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@ApplicationPath("/api/")
public class AppConfig extends Application {

}
```

Zasoby

Konfiguracja zasobów oraz definiowanie routingu odbywa się za pomocą adnotacji::

- `@Path` - relatywny adres, pod którym będzie dostępna operacja, żądania HTTP:
- `@GET`, `@POST`, `@PUT`, `@DELETE`, `@PATCH`, `@HEAD` – rodzaje żądań HTTP,
- `@PathParam` - parametry pobierane z URI żądania jako elementy URI,
- `@QueryParam` - parametry pobierane z URI żądania jako parametry żądania,
- `@Consumes`, `@Produces` - specyfikacja typu MIME.

Adresowanie usług:

- bazowy adres `@ApplicationPath("base")`,
- globalny adres dla całej klasy poprzez `@Path("global")` na klasie,
- relatywny adres dla metody poprzez `@Path("relative")` na metodzie.

Zasoby - przykład

Przykładowe żądanie `GET /api/books`.

```
public class BookController {

    @Path("books")
    @GET
    public List<Book> getBooks() {
        // ...
    }

}
```

```
@Path("books")
public class BookController {

    @GET
    public List<Book> getBooks() {
        // ...
    }

}
```

Odwzorowanie parametrów - ścieżka

Sposób odwzorowania parametrów żądania HTTP na parametry metody kontrolera definiują adnotacje:

- parametry w URI: `@PathParam("id")`,
- przykładowe żądanie: `GET /api/books/1`.

```
@Path("books")
public class BookController {

    @Path("/{id}")
    @GET
    public Book getBook(@PathParam("id") Integer id) {
        // ...
    }

}
```

Odwzorowanie parametrów - żądanie

Sposób odwzorowania parametrów żądania HTTP na parametry metody kontrolera definiują adnotacje:

- parametry w query string: `@QueryParam("q")`,
- przykładowe żądanie: `GET /books?q=Ender`,
- dla parametrów opcjonalnych można określić wartość domyślną: `@DefaultValue("Ender")`.

```
@Path("books")
public class BookController {

    @GET
    public List<Book> search(@QueryParam("q") @DefaultValue("Ender") String query) {
        // ...
    }

}
```

Odwzorowanie parametrów - formularz

Sposób odwzorowania parametrów żądania HTTP na parametry metody kontrolera definiują adnotacje:

- parametry z formularza: @FormParam("login"),
- dla danych w formacie application/x-www-form-urlencoded.

```
POST /api/token HTTP/1.1
Host: localhost:8080
Content-Type: application/x-www-form-urlencoded
login=admin&password=admin
```

```
public class AuthController {

    @Path("/token")
    @POST
    public Response token(
            @FormParam("login") String login,
            @FormParam("password") String pass
    ) {
        // ...
    }

}
```

Odwzorowanie parametrów - ciało żądania

Sposób odwzorowania parametrów:

- Dane z ciała żądania: `@Consumes`.

```
POST /api/books/ HTTP/1.1
Host: localhost:8080
Content-Type: application/json

{
    "title": "Teatr cieni",
    "cover": "SOFT",
    "publishDate": 1297724400000,
}
```

```java
@Path("books")
public class BookController {

    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    public Response postBook(Book book) {
        // ...
    }

}
```

Wartości zwracane - typ surowy

Obiekty zwracane z metod kontrolerów są konwertowane zgodnie z adnotacją `@Produces`.

```
@Path("books")
public class BookController {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Book> getBooks() {
        // ...
    }

}
```

Wartości zwracane z kontrolerów

Dodatkową kontrolę nad odpowiedzią HTTP można uzyskać zwracając obiekt klasy `Response`:

- `Response.ok(entity).build()`:
  - Odpowiedź 200,
  - w ciele odpowiedzi obiekt entity przekonwertowany do wskazanego formatu;
- `Response.created(URI.create("/book/29")).build()`:
  - Odpowiedź 201,
  - nagłówek Location z zawartością parametru.
- `Response.accepted().build()`:
  - odpowiedź 202;
- `Response.seeOther(URI.create("/new/uri")).build()`:
  - odpowiedź typu redirect,
  - nagłówek Location z zawartością parametru;
- `Response.status(Response.Status.NOT_FOUND).build()`:
  - Odpowiedź HTTP z wybranym kodem;
- `Response.header("X-My-Header", "...").build();`:
  - ustawianie nagłówków odpowiedzi.

Wartości zwracane - typ opakowujący

```
@Path("books")
public class BookController {

    @Path("/{id}")
    @GET
    public Response getBook(@PathParam("id") Integer id) {
        Book book = service.findBook(id);
        return book != null ?
                Response.ok(book).build() :
                Response.status(NOT_FOUND).build();
    }

}
```

SOP vs CORS

SOP - Same Origin Policy:

- origin (pochodzenie, czyli protokół, host i port) definiuje jednoznacznie aplikację,
- różne aplikacje (np.: strony) nie mogą wymieniać się treścią:
  - linkowanie obrazków,
  - wysyłanie formularza na inny serwer;
- chroni m.in. przed atakami XSS (Cross-site scripting) i CSRF (Cross-site request forgery),
- domyślne zachowanie przeglądarki,
- blokuje żądania AJAX między aplikacjami.

CORS - Cross-Origin Resource Sharing:

- różne aplikacje mogą wymieniać się treścią,
- definiowane w postaci nagłówków odpowiedzi po stronie serwera,
- pozwala m.in. na:
  - aplikacje JavaScript komunikujące się z API na innym serwerze,
  - systemy SSO dla kilku aplikacji.

CORS - konfiguracja

```
@Provider
public class CorsFilter implements ContainerResponseFilter {

    @Override
    public void filter(ContainerRequestContext requestContext,
                       ContainerResponseContext responseContext) throws IOException {
        responseContext.getHeaders().add(
                "Access-Control-Allow-Origin", "*"
        );
        responseContext.getHeaders().add(
                "Access-Control-Allow-Credentials", "true"
        );
        responseContext.getHeaders().add(
                "Access-Control-Allow-Headers", "origin, content-type, accept, authorization"
        );
        responseContext.getHeaders().add(
                "Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS, HEAD"
        );
    }
}
```

Pomocne źródła

Specyfikacja:

- https://jakarta.ee/specifications/restful-ws/3.1/jakarta-restful-ws-spec-3.1

Uzupełnienie informacji:

- https://eclipse-ee4j.github.io/jakartaee-tutorial/#building-restful-web-services-with-jakarta-rest
- https://www.baeldung.com/jax-rs-spec-and-implementations
- https://www.baeldung.com/rest-api-jax-rs-vs-spring
- https://www.baeldung.com/jax-rs-response
- https://www.baeldung.com/jersey-jax-rs-client